Lab 1.3.1: Examine RAG Solutions

Scenario

Retrieval-Augmented Generation (RAG) is a technique that combines the reasoning ability of language models with the precision of external knowledge sources. Instead of relying only on the model's internal training data, RAG systems pull in relevant chunks of documents at query time, grounding answers in up-to-date or domain-specific context.

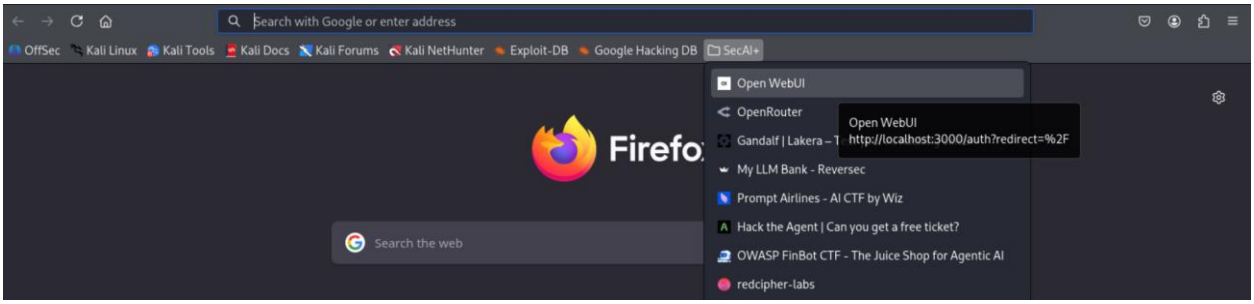
In this lab, you'll practice how retrieval settings like chunk size, overlap, and Top-K affect results and why large documents can overwhelm a model's context window. By the end, you'll understand not just how to configure RAG, but also its strengths, limitations, and tradeoffs.

Your Mission

- Learn about model context limits and full document retrieval.
- Adjust chunk size, overlap, and retrieval parameters to see how these settings influence precision, recall, and overall answer quality.

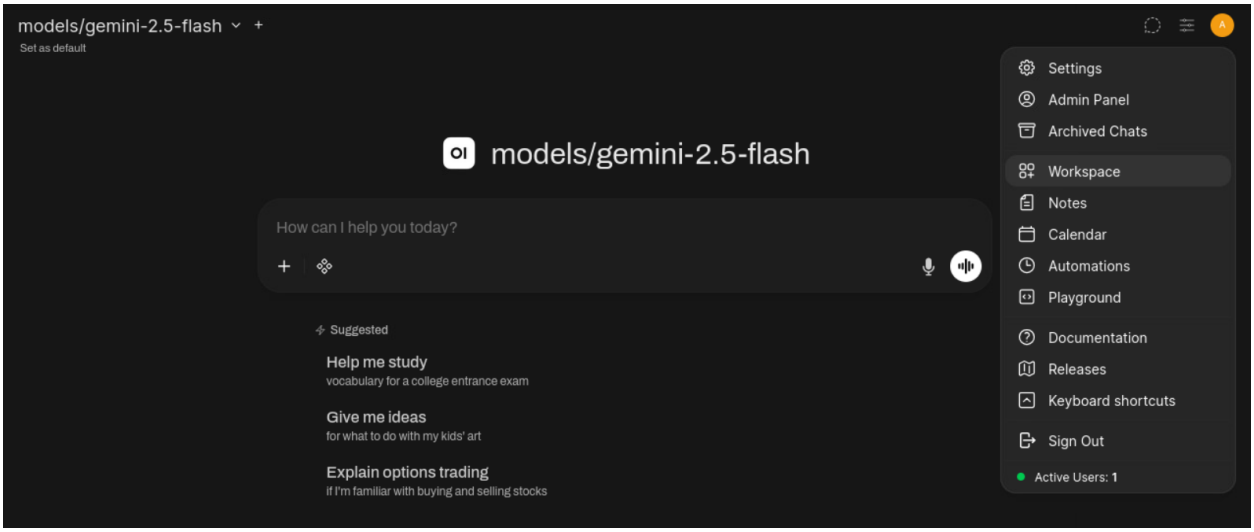
Full-Document Retrieval

Step 1: From the desktop, open **Firefox** and click on SecAI+ bookmark tab and select Open WebUI to sign into **Open WebUI**.



Step 2: Select **Workspace** from the left Username icon (It is present on the right hand side too), then select the **Knowledge** tab.

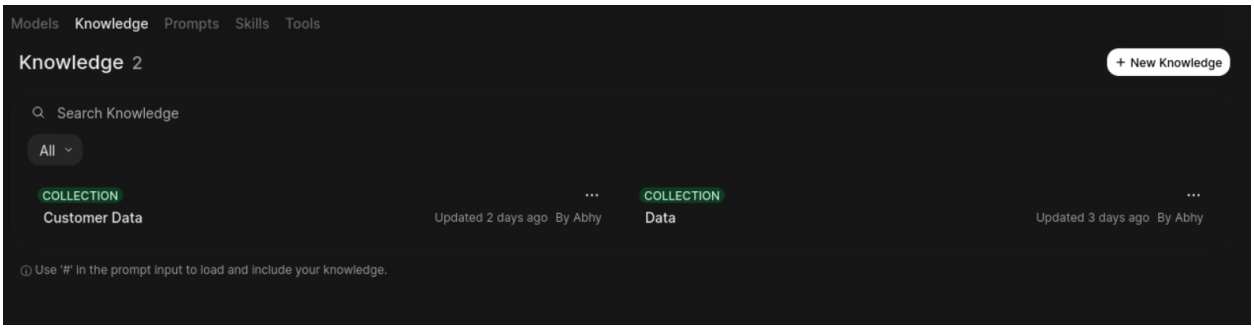
- If the menu is not visible, select the three horizontal lines to the left of the model name to activate the menu.



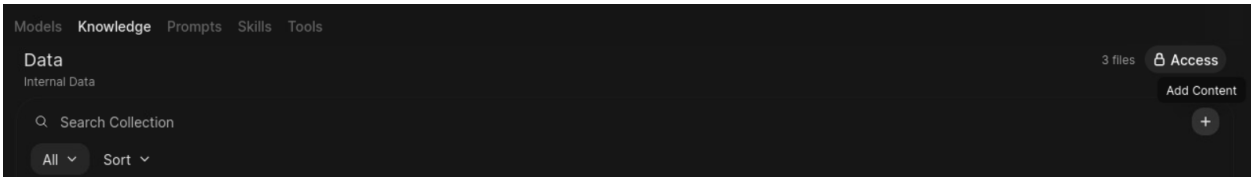
Step 3: From the Knowledge tab, select the **Data** collection. First time you won't find data collection, so you will have to click on + New Knowledge and upload the required files:
File 1: Development.md
File 2: api-endpoints.md

All files for this lab are available in the github repository in the folder **Labfiles 1.3**

Download all 3 files and save it in the directory home/kali/Labfiles 1.3



Step 4: From the Data collection, select **+** to the right of the page, then select **Upload files**.



Step 5: Select the **rag.md** file in **Labfiles 1.3**, then select **Open**.

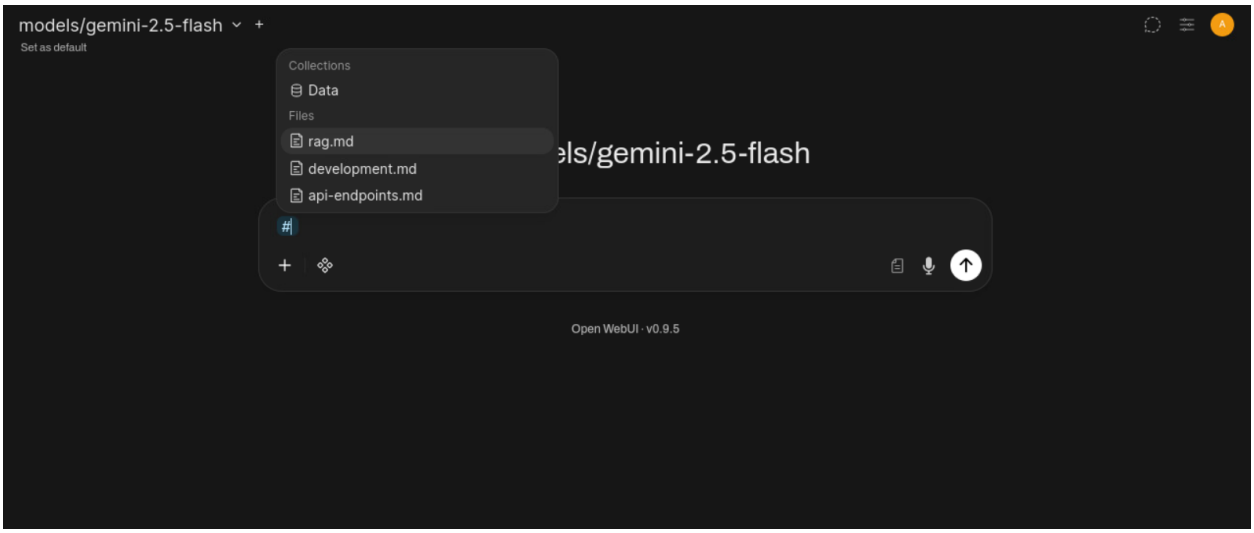
> Insights: There should now be three files in the Data collection.

Step 6: Select the **rag.md** file from the collection on the right.

> Insights: When you select a file in the Knowledge workspace, you see the text as the model sees it after extraction. With full document retrieval, everything here is sent into the prompt at once. That means more tokens, more context space consumed, and more chance the model focuses on irrelevant parts instead of just what you asked for.

Step 7: With the data uploaded, select **New Chat** from the top left of OpenWebUI.

Step 8: In a new chat with **gemini flash 2.5**, enter **#** to bring up the knowledge store. Select **rag.md** from the list.



Step 9: In the chat, enter the following prompt and note the response:

```
Prompt 1: What is the maximum default context length for Ollama?
```

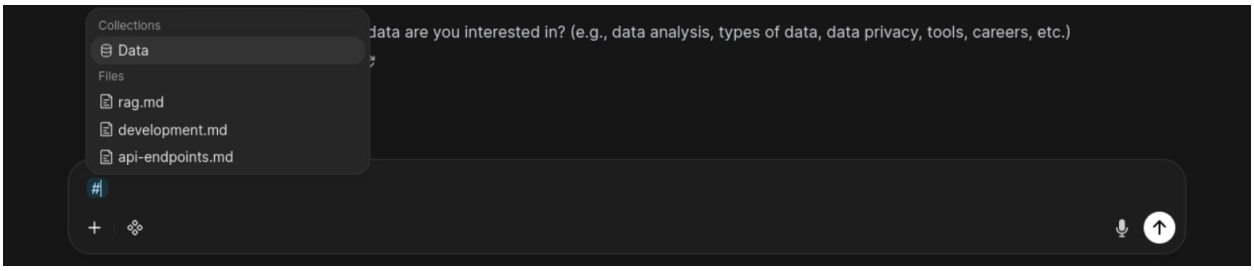
Step 10: In the same chat, enter the following prompt and note the response:

```
Prompt 2: Can this be increased?
```

> Insights: Each response will show the sources used to generate it. In this case, the rag.md entry below each of the responses indicates that it's the data source being used.

Step 11: Return to the **Data collection** in the Knowledge tab and select the **rag.md** file to compare the results to the model's response.

Step 12: Start a new chat with **gemini** and enter **#data**, then press Enter to bring up the Data Collection knowledge store, or select **Data Collection** from the list.



> Insights: Prompting against the entire data collection lets you ask broader, comparative questions across multiple documents. Querying a single file keeps the scope narrow and precise. Together, these modes let you zoom out for trends or zoom in for detail, depending on the task.

Step 13: In the chat, enter the following prompt and note the results:

```
Prompt 3: What are the benefits of using RAG?
```

> Insights: Full-doc retrieval quickly consumes the model's context window, especially with multiple files.
> Even if it doesn't error, huge prompts leave less room for your actual query and answer, forcing the model to drop or compress information. Chunking ensures only the most relevant parts of each file are included, keeping responses accurate and efficient.

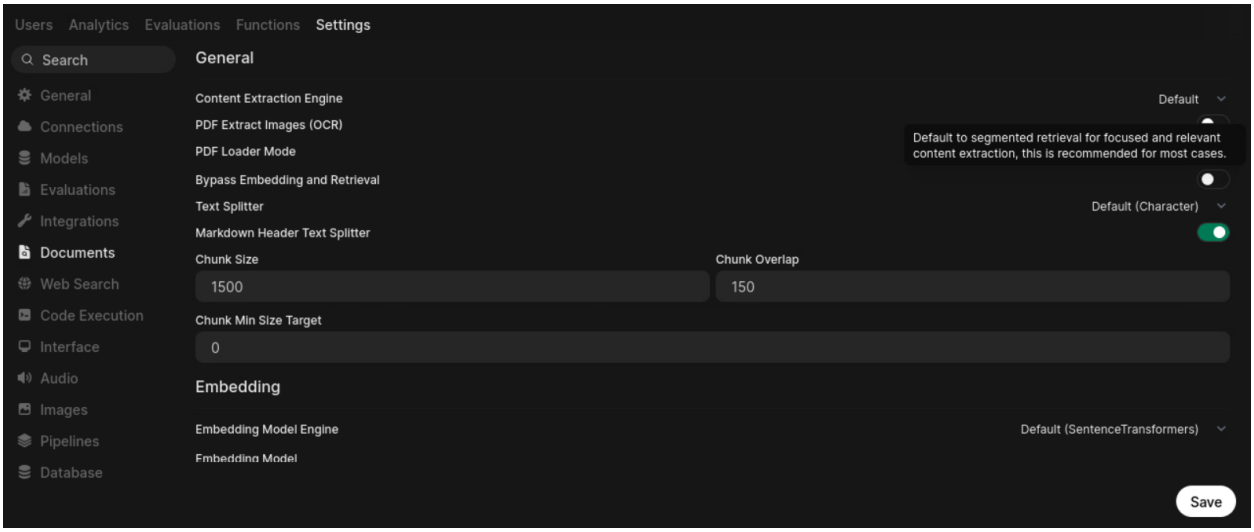
1.3.2 Chunking, Overlap, and Retrieval Settings

Full-doc retrieval showed the limits of dumping an entire file into context. Now you'll explore how RAG breaks documents into smaller “chunks” for retrieval. By adjusting chunk size, overlap, and retrieval parameters, you'll see how these settings change the balance between precision, completeness, and noise in the answers.

Step 1: From Open WebUI, select **User** from the bottom-left menu, then select **Settings**.

Step 2: Select **Admin Settings**, then select **Documents** from the left menu.

Step 3: Under **General**, toggle **Bypass Embedding and Retrieval** off and select **Save** at the lower right.



Insights: This disables full-doc retrieval and enables embedding and retrieval settings below.

Step 4: Start a new chat with **gemini** and enter **#** to bring up the knowledge store. Select the **Data** collection from the list.

Step 5: In the chat, enter the following prompt and note the results:

```
Prompt 1: What are the benefits of using RAG?
```

> Insights: Now that chunking is enabled, the data collection can be processed within the model's context limits. However, with the default settings (Chunk Size = 1000, Chunk Overlap = 100), the model still fails to return the information we're looking for — even though we know it's present in rag.md.

Step 6: Return to the RAG settings by selecting **User > Settings > Admin Settings > Documents**.

Step 7: Under the **General** section, change the **Chunk Size** to **300** and the **Chunk Overlap** to **80**, then select **Save**.

> Insights: Chunk size controls how much text goes into each piece: larger values provide more context but risk pulling in irrelevant details, while smaller values are more precise but may fragment ideas.
> Overlap determines how much chunks share with each other: higher overlap keeps continuity between chunks but adds redundancy, while lower overlap is more efficient but can cut off context mid-thought.

Step 8: Return to the **Data** collection by selecting **Workspace > Knowledge > Data**.

Step 9: Remove **rag.md**, **development.md**, and **api-endpoints.md** from the collection.

> Action: You'll need to hover over each file and select the ⨯ icon to remove it.

Step 10: Re-upload all 3 files to the collection.

> Insights: Chunk size and overlap control how a document is split into pieces during ingestion. Once a file is uploaded, those chunks are fixed. Changing the chunking settings doesn't retroactively rebuild them, so the files must be deleted and re-uploaded for the new settings to take effect.

Step 11: Start a new chat with **gemini** and enter **#** to bring up the knowledge store. Select the **Data** collection from the list.

Step 12: In the chat, enter the following prompt and note the results:

```
Prompt 2: What are the benefits of using RAG?
```

> Insights: With the new chunking settings, the responses should now be much more accurate. Different file types and formats often benefit from different chunking configurations, so when applying RAG to your own data, some experimentation may be needed to find the settings that work best.

Step 13: Start a new chat with **gemini** and enter **#** to bring up the knowledge store. Select **rag.md** from the list.

Step 14: In the chat, enter the following prompt and note the results:

```
Prompt 3: List the steps under ## Detailed Instructions.
```

> Insights: The model will likely have an issue finding the mentioned section.

Step 15: Return to the RAG settings by selecting **User > Settings > Admin Settings > Documents**.

Step 16: Under the **Retrieval** section, set the **Top K** value to **10** and select **Save**.

> Insights: Because Top-K is a retrieval setting, files don't need to be re-uploaded when it's changed. This option affects how many chunks are pulled at query time, not how the data is stored during upload.

Step 17: Start a new chat with **gemini** and enter **#** to bring up the knowledge store. Select **rag.md** from the list.

Step 18: In the chat, enter the following prompt and note the results:

```
Prompt 4: List the steps under ## Detailed Instructions.
```

> Insights: Increasing Top K helped retrieval find the right section by pulling in more chunks, but because the chunk size is still very small (300 with overlap 80), each chunk only contains a

> sliver of the text.
> That's why the answer is still incomplete — the model can see where the information is, but not enough of it at once.

Step 19: Return to the RAG settings and set the **Chunk Size** to **1500** and **Chunk Overlap** to **150**.

Step 20: Return to the **Data** collection, delete **rag.md** and then re-upload it.

> Insights: Because files are chunked at upload, leaving the ones set to 300 tokens in place means the same collection can hold documents with different chunking configurations.

Step 21: Start a new chat with **gemini** and enter **#** to bring up the knowledge store. Select **rag.md** from the list.

Step 22: In the chat, enter the following prompt and note the results:

```
Prompt 5: List the steps under ## Detailed Instructions.
```

> Insights: With the larger chunk size and overlap, the model should now return a more complete answer. You can verify its accuracy by viewing rag.md in the Data collection.